CPU Scheduling – Detailed Notes

1. What is CPU Scheduling?

CPU Scheduling is the process of selecting a process from the **ready queue** and allocating the CPU to it for execution.

In a multiprogramming operating system, several processes may be present in memory at the same time. However, only one process can use a single CPU at a particular instant. Therefore, the operating system must decide **which process should execute next**.

The component responsible for this decision is called the **CPU Scheduler**.

---

2. CPU Scheduler (Short-Term Scheduler)

The **CPU Scheduler**, also called the **Short-Term Scheduler**, selects one process from the processes available in the **ready queue**.

Main function:

Select a process from the ready queue and allocate the CPU to that process.

Ready Queue

The **ready queue** contains processes that are:

- Ready to execute.
- Waiting for CPU allocation.

Example

Suppose the ready queue contains:

Process	Status
P1	Ready
P2	Ready
P3	Ready

The CPU scheduler selects one process, for example **P2**, based on a scheduling algorithm such as FCFS, SJF, or Round Robin.

---

3. Dispatcher

Once the CPU Scheduler selects a process, the **Dispatcher** gives control of the CPU to that selected process.

The Dispatcher performs the following operations:

1. Context Switching

The operating system saves the state of the currently running process and loads the state of the newly selected process.

The state may include:

- Program counter
- CPU registers
- Process state
- Other execution information

This process is called **Context Switching**.

2. Switching to User Mode

The CPU switches from **kernel mode** to **user mode** so that the selected user process can execute.

3. Jumping to the Correct Location

The Dispatcher transfers control to the correct instruction of the selected program so that the process can continue execution from where it previously stopped.

Dispatcher Latency

The time required by the Dispatcher to stop one process and start another process is called **Dispatcher Latency**.

---

4. Process State Transitions and CPU Scheduling

A process can move between different states during execution.

The main process states are:

- **New**
- **Ready**
- **Running**
- **Waiting**

- **Terminated**

The CPU Scheduler can be invoked during certain state transitions.

Four Situations When CPU Scheduling Can Occur

1. Running → Waiting

A process moves from the **Running state** to the **Waiting state** when it has to wait for an event.

For example:

- Waiting for keyboard input
- Waiting for disk I/O
- Waiting for a file to be read

Since the process cannot continue execution, the CPU scheduler selects another process.

Example:

P1 Running → Requests Disk I/O → Waiting

The CPU can now be allocated to another ready process.

---

2. Running → Ready

A process may be moved from the **Running state** back to the **Ready state**.

This usually happens when:

- Its time quantum expires.
- A higher-priority process arrives.

The current process is interrupted, and another process may be selected.

Example:

P1 Running → Time Quantum Expires → Ready

---

3. Waiting → Ready

A process moves from the **Waiting state** to the **Ready state** when the event it was waiting for is completed.

For example:

P1 Waiting → I/O Completed → Ready

The scheduler may decide to allocate the CPU to this newly ready process.

---

4. Running → Terminated

A process enters the **Terminated state** after completing its execution.

P1 Running → Execution Completed → Terminated

The CPU Scheduler then selects another process from the ready queue.

---

5. Non-Preemptive Scheduling

In **Non-Preemptive Scheduling**, once a process gets the CPU, it continues executing until:

1. It completes its execution, or
2. It voluntarily moves to the waiting state.

The operating system does **not forcibly take the CPU away** from the process.

Therefore, the following scheduling points are non-preemptive:

Running → Waiting

The process releases the CPU because it needs to wait for an event.

Running → Terminated

The process releases the CPU because it has completed execution.

Important Point

In non-preemptive scheduling, the currently running process keeps the CPU until it finishes or blocks.

Examples

- First Come First Serve (FCFS)
- Non-preemptive Shortest Job First (SJF)
- Non-preemptive Priority Scheduling

---

6. Preemptive Scheduling

In **Preemptive Scheduling**, the operating system can interrupt a running process and take the CPU away before the process finishes.

The CPU can then be given to another process.

The following scheduling points are generally associated with preemptive scheduling:

Running → Ready

The currently running process is interrupted and returned to the ready queue.

This may happen when:

- The time quantum expires.
- A higher-priority process arrives.

Waiting → Ready

When a process becomes ready after completing an event, the scheduler may decide that it should immediately execute instead of the currently running process.

Important Point

In preemptive scheduling, a running process can be forced to release the CPU before completing its execution.

Examples

- Round Robin Scheduling
- Shortest Remaining Time First (SRTF)
- Preemptive Priority Scheduling

---

7. Difference Between Preemptive and Non-Preemptive Scheduling

Preemptive Scheduling	**Non-Preemptive Scheduling**
CPU can be taken away from a running process.	CPU cannot be forcibly taken away.
Process may be interrupted before completion.	Process continues until completion or waiting state.
Provides better response for interactive systems.	Simpler to implement.
Requires more context switching.	Requires fewer context switches.

Preemptive Scheduling	Non-Preemptive Scheduling
Example: Round Robin	Example: FCFS

---

8. Scheduling Criteria

Different CPU scheduling algorithms are compared using several performance criteria.

The main scheduling criteria are:

1. CPU Utilization
2. Throughput
3. Turnaround Time
4. Waiting Time
5. Response Time

8.1 CPU Utilization

CPU Utilization refers to the percentage of time during which the CPU is actively executing processes.

The objective is to keep the CPU busy as much as possible.

Formula

$$\text{CPU Utilization} = \frac{\text{CPU Busy Time}}{\text{Total Time}} \times 100$$

Example

If the CPU is busy for 90 seconds out of 100 seconds:

$$CPU\ Utilization = \frac{90}{100} \times 100 = 90\%$$

Goal

Maximize CPU Utilization.

8.2 Throughput

Throughput is the number of processes completed per unit time.

Example

If 20 processes are completed in 10 seconds:

$$Throughput = \frac{20}{10} = 2$$

Therefore:

Throughput = 2 processes per second.

Goal

Maximize Throughput.

A higher throughput means that more processes are completed in less time.

8.3 Turnaround Time

Turnaround Time (TAT) is the total time taken by a process from its arrival until its completion.

It includes:

- Waiting time
- CPU execution time
- Other delays

Formula

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$$

Example

Suppose:

- Arrival Time = 2 ms
- Completion Time = 12 ms

Then:

$$TAT = 12 - 2 = 10 \text{ ms}$$

Goal

Minimize the average Turnaround Time.

8.4 Waiting Time

Waiting Time (WT) is the total time a process spends waiting in the ready queue for CPU allocation.

Formula

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

Example

Suppose:

- Turnaround Time = 10 ms
- Burst Time = 6 ms

Then:

$$WT = 10 - 6 = 4 \text{ ms}$$

The process waited for **4 ms** in the ready queue.

Goal

Minimize the average Waiting Time.

8.5 Response Time

Response Time is the time from when a process or request arrives until it receives its **first response**.

It does not mean the total time required to complete the process.

Formula

$$\text{Response Time} = \text{First CPU Allocation Time} - \text{Arrival Time}$$

Example

A process arrives at:

Time = 2 ms

It first gets the CPU at:

Time = 7 ms

Therefore:

$$Response\ Time = 7 - 2 = 5 \text{ ms}$$

Goal

Minimize Response Time.

Response time is especially important in:

- Interactive systems
- Time-sharing systems
- Online applications

---

9. Optimization Criteria

An ideal CPU scheduling algorithm should satisfy the following:

1. Maximize CPU Utilization

Keep the CPU busy as much as possible.

2. Maximize Throughput

Complete more processes in a given amount of time.

3. Minimize Average Turnaround Time

Reduce the total time required for processes to complete.

4. Minimize Average Waiting Time

Reduce the amount of time processes spend waiting in the ready queue.

5. Minimize Average Response Time

Provide the first response to processes as quickly as possible.

---

10. Important CPU Scheduling Terms

Arrival Time (AT)

Arrival Time is the time at which a process enters the ready queue.

Example

If process P1 enters the ready queue at time 2:

$$AT = 2$$

Burst Time (BT)

Burst Time is the amount of CPU execution time required by a process.

It represents how long the process needs the CPU.

Example

If P1 needs the CPU for 5 ms:

$$BT = 5 \text{ ms}$$

Completion Time (CT)

Completion Time is the time at which a process completely finishes its execution.

Example

If P1 finishes at time 10:

$$CT = 10$$

Turnaround Time (TAT)

The total time from process arrival until process completion.

Formula

$$\boxed{TAT = CT - AT}$$

Where:

- CT = Completion Time
- AT = Arrival Time

Waiting Time (WT)

The total time a process spends waiting for the CPU.

Formula

$$\boxed{WT = TAT - BT}$$

Where:

- TAT = Turnaround Time
- BT = Burst Time

---

11. Simple Example

Consider the following process:

Process	Arrival Time	Burst Time	Completion Time
P1	2 ms	5 ms	10 ms

Step 1: Calculate Turnaround Time

$$TAT = CT - AT$$
$$TAT = 10 - 2 = 8 \text{ ms}$$

Step 2: Calculate Waiting Time

$$WT = TAT - BT$$
$$WT = 8 - 5 = 3 \text{ ms}$$

Therefore:

Process	AT	BT	CT	TAT	WT
P1	2	5	10	8	3

---

12. Quick Formula Summary

$$\boxed{Turnaround\ Time = Completion\ Time - Arrival\ Time}$$
$$\boxed{Waiting\ Time = Turnaround\ Time - Burst\ Time}$$
$$\boxed{Response\ Time = First\ CPU\ Allocation\ Time - Arrival\ Time}$$

---

13. One-Line Exam Summary

CPU Scheduling is the process of selecting a process from the ready queue and allocating the CPU to it. The **CPU Scheduler** selects the process, while the **Dispatcher** transfers CPU control to it. Scheduling can be **preemptive or non-preemptive**, and scheduling algorithms are evaluated using CPU utilization, throughput, turnaround time, waiting time, and response time.

First-Come, First-Served (FCFS) Scheduling

1. Definition

First-Come, First-Served (FCFS) is the simplest CPU scheduling algorithm in which the process that arrives first in the ready queue gets the CPU first.

It follows the **FIFO (First-In, First-Out)** principle.

First process to arrive → First process to execute

---

2. Main Characteristics of FCFS

1. Single FIFO Ready Queue

All processes are placed in a single **ready queue** according to their arrival time.

The process that enters the queue first is executed first.

Example:

Ready Queue: P1 → P2 → P3

Execution order:

P1 → P2 → P3

---

2. Non-Preemptive Scheduling

FCFS is a **non-preemptive scheduling algorithm**.

Once a process gets the CPU, it continues executing until:

- It completes its execution, or
- It moves to the waiting state.

The CPU cannot be forcibly taken away from the process.

---

3. Simple to Implement

FCFS is easy to understand and implement because it uses a simple FIFO queue.

---

4. Not Suitable for Time-Sharing Systems

FCFS is generally not suitable for **time-sharing systems** because a long process can occupy the CPU for a long time while other processes wait.

Interactive users may experience a long delay.

---

5. Average Waiting Time Depends on Arrival Order

The average waiting time in FCFS depends heavily on the order in which processes arrive.

A long process arriving first can significantly increase the waiting time of shorter processes.

Example 1

Consider the following processes:

Process	Burst Time
P1	24
P2	3
P3	3

Suppose the processes arrive in the order:

P1 → P2 → P3

Since FCFS follows the arrival order, the execution order is also:

P1 → P2 → P3

Gantt Chart

| P1 | P2 | P3 |

0 24 27 30

Waiting Time Calculation

P1

P1 executes immediately.

$$WT(P1) = 0$$

P2

P2 must wait until P1 completes.

$$WT(P2) = 24$$

P3

P3 must wait until P1 and P2 complete.

$$WT(P3) = 24 + 3 = 27$$

Process	Waiting Time
P1	0

Process	Waiting Time
P2	24
P3	27

Average Waiting Time

$$\text{Average WT} = \frac{0 + 24 + 27}{3}$$

$$\boxed{\text{Average Waiting Time} = 17}$$

Example 2

Now suppose the processes arrive in a different order:

P2 → P3 → P1

The FCFS execution order becomes:

P2 → P3 → P1

Gantt Chart

| P2 | P3 | P1 |

0 3 6 30

Waiting Time Calculation

P2

P2 executes first.

$$WT(P2) = 0$$

P3

P3 waits for P2.

$$WT(P3) = 3$$

P1

P1 waits for P2 and P3.

$$WT(P1) = 3 + 3 = 6$$

Process	Waiting Time
P1	6
P2	0
P3	3

Average Waiting Time

$$\text{Average WT} = \frac{6 + 0 + 3}{3}$$

$$\boxed{\text{Average Waiting Time} = 3}$$

Important Observation

The same three processes have very different average waiting times depending on their arrival order:

Arrival Order	Average Waiting Time
P1 → P2 → P3	17
P2 → P3 → P1	3

Therefore:

In FCFS scheduling, the average waiting time strongly depends on the order in which processes enter the ready queue.

Convoy Effect

The main disadvantage of FCFS scheduling is the **Convoy Effect**.

Definition

The **Convoy Effect** occurs when a long process gets the CPU before several short processes.

The short processes must wait behind the long process, similar to a group of vehicles waiting behind a slow vehicle.

Example

Suppose:

P1 = 20 ms

P2 = 2 ms

P3 = 1 ms

If P1 arrives first:

P1 → P2 → P3

Both P2 and P3, although very short, must wait for P1 to finish.

| P1 | P2 | P3 |

0 20 22 23

This causes a large waiting time for short processes.

Convoy Effect: Short processes wait behind a long process.

Advantages of FCFS

1. Simple and easy to understand.
2. Easy to implement using a FIFO queue.
3. Processes are served fairly according to arrival order.
4. No starvation because every process will eventually get the CPU.
5. Low scheduling overhead.

Disadvantages of FCFS

1. Can produce high average waiting time.
2. Average waiting time depends on process arrival order.
3. Suffers from the **convoy effect**.
4. Not suitable for time-sharing systems.
5. A long process can make short processes wait for a long time.
6. Poor response time for interactive processes.

Problem 2 – FCFS Scheduling

The table in your notes appears to contain:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	2
P3	3	4
P4	5	5

Since the processes arrive in the order **P1 → P2 → P3 → P4**, FCFS executes them in the same order.

Step 1: Draw the Gantt Chart

P1

- Arrives at 0
- Executes for 8 units
- Completes at 8

P2

- Arrives at 1
- Starts at 8
- Burst time = 2
- Completes at 10

P3

- Arrives at 3
- Starts at 10
- Burst time = 4
- Completes at 14

P4

- Arrives at 5
- Starts at 14
- Burst time = 5
- Completes at 19

| P1 | P2 | P3 | P4 |

0 8 10 14 19

Step 2: Calculate Completion Time

Process	AT	BT	CT
P1	0	8	8
P2	1	2	10
P3	3	4	14
P4	5	5	19

Step 3: Calculate Turnaround Time

$$TAT = CT - AT$$

Process	CT	AT	TAT
P1	8	0	8
P2	10	1	9
P3	14	3	11
P4	19	5	14

Step 4: Calculate Waiting Time

$$WT = TAT - BT$$

Process	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	1	2	10	9	7
P3	3	4	14	11	7
P4	5	5	19	14	9

Average Waiting Time

$$\text{Average WT} = \frac{0 + 7 + 7 + 9}{4}$$

$$\boxed{\text{Average Waiting Time} = 5.75}$$

Final FCFS Summary

FCFS (First-Come, First-Served) is a **non-preemptive CPU scheduling algorithm** that follows the **FIFO principle**. The process that arrives first gets the CPU first. It is simple to implement but may result in high waiting time and suffers from the **convoy effect**, where short processes wait behind long processes.

Shortest-Job-First (SJF) Scheduling – Detailed Notes

1. Definition

Shortest-Job-First (SJF) is a CPU scheduling algorithm in which the CPU is allocated to the process having the **smallest CPU burst time**.

The process requiring the shortest CPU time is executed first.

If two or more processes have the same burst time, **FCFS** is generally used to break the tie.

---

2. Main Idea of SJF

Each process is associated with its **next CPU burst time**.

The scheduler compares the burst times of all ready processes and selects the process with the **shortest burst time**.

Example

Process	Burst Time
P1	6
P2	2
P3	4

If all processes are ready at the same time, the execution order will be:

P2 → P3 → P1

Because:

$$2 < 4 < 6$$

---

3. Types of SJF Scheduling

SJF has two forms:

1. Non-Preemptive SJF

Once a process gets the CPU, it continues execution until it completes.

It cannot be interrupted even if another process with a shorter burst time arrives.

---

2. Preemptive SJF

A running process can be interrupted if a newly arriving process has a shorter remaining CPU burst time.

Preemptive SJF is also called:

Shortest-Remaining-Time-First (SRTF)

The process with the shortest **remaining execution time** gets the CPU.

---

4. Non-Preemptive SJF

In **Non-Preemptive SJF**, when the CPU becomes free:

1. Check all processes in the ready queue.
2. Select the process with the shortest burst time.
3. Allocate the CPU to that process.
4. Once execution starts, the process cannot be interrupted.

Important Point

Even if a shorter process arrives later, the currently running process continues until completion.

Example: Non-Preemptive SJF

Consider:

Process	Arrival Time	Burst Time
P1	0	7

Process	Arrival Time	Burst Time
P2	2	4
P3	4	1
P4	5	4

Step 1: Time = 0

Only **P1** has arrived.

Therefore, P1 gets the CPU.

Time 0 → P1 starts

P1 has burst time = 7.

Since this is **non-preemptive SJF**, P1 continues until completion.

P1 executes from 0 to 7

Even though P2, P3, and P4 arrive while P1 is executing, P1 cannot be interrupted.

Step 2: Time = 7

P1 completes.

The ready queue now contains:

Process	Burst Time
P2	4
P3	1
P4	4

The shortest burst time is P3 = 1.

Therefore, P3 executes next.

P3 executes from 7 to 8

Step 3: Time = 8

Remaining processes:

Process	Burst Time
P2	4
P4	4

Both have the same burst time.

Therefore, FCFS is used to break the tie.

P2 arrived before P4, so P2 executes first.

P2 executes from 8 to 12

Step 4: Time = 12

Only P4 remains.

P4 executes from 12 to 16

Gantt Chart

P1	P3	P2	P4

0 7 8 12 16

Waiting Time Calculation

P1

$$WT = Start\ Time - Arrival\ Time$$
$$WT = 0 - 0 = 0$$

P2

$$WT = 8 - 2 = 6$$

P3

$$WT = 7 - 4 = 3$$

P4

$$WT = 12 - 5 = 7$$

Therefore:

Process	Arrival Time	Burst Time	Start Time	Completion Time	Waiting Time
P1	0	7	0	7	0
P2	2	4	8	12	6
P3	4	1	7	8	3
P4	5	4	12	16	7

Average Waiting Time

$$\frac{0+6+3+7}{4} = \frac{16}{4} = \boxed{4}$$

Note: The waiting-time values shown in the provided notes appear inconsistent with the given arrival times. Using the standard formula $WT = Start\ Time - Arrival\ Time$, the average waiting time is **4 units**.

---

5. Why Non-Preemptive SJF is Optimal

When **all processes are available at the same time**, Non-Preemptive SJF produces the **minimum average waiting time**.

This is because short processes finish early instead of waiting behind long processes.

Example

Processes:

Process	Burst Time
P1	10
P2	2
P3	1

If P1 runs first:

P1 → P2 → P3

Short processes must wait a long time.

SJF chooses:

P3 → P2 → P1

This reduces the average waiting time.

---

6. Preemptive SJF / SRTF

Shortest-Remaining-Time-First (SRTF) is the preemptive version of SJF.

The CPU is always given to the process with the **smallest remaining burst time**.

Whenever a new process arrives:

1. Compare its burst time with the remaining time of the currently running process.
2. If the new process has a shorter execution time, interrupt the current process.
3. Give the CPU to the new process.

Example: Preemptive SJF (SRTF)

Consider:

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Time = 0

Only P1 is available.

P1 starts execution.

P1 runs from 0 to 2.

Remaining time of P1:

$$7 - 2 = 5$$

Time = 2

P2 arrives.

Comparison:

Process	Remaining Time
P1	5
P2	4

P2 has the shorter remaining time.

Therefore, **P1 is preempted** and P2 gets the CPU.

P1: 0 → 2

P2 starts at 2

Time = 4

P3 arrives.

P2 has executed for 2 units.

Remaining time of P2:

$$4 - 2 = 2$$

Comparison:

Process	Remaining Time
P1	5
P2	2
P3	1

P3 has the shortest remaining time.

Therefore, P2 is preempted and P3 gets the CPU.

Time = 5

P3 completes exactly at time 5.

P4 also arrives.

Ready processes are:

Process	Remaining Time
P1	5
P2	2
P4	4

P2 has the shortest remaining time.

Therefore, P2 executes.

P2: 5 → 7

P2 completes.

Time = 7

Ready processes:

Process	Remaining Time
P1	5
P4	4

P4 has the shorter remaining time.

Therefore:

P4: 7 → 11

Time = 11

Only P1 remains.

P1: 11 → 16

Final Gantt Chart

| P1 | P2 | P3 | P2 | P4 | P1 |

0 2 4 5 7 11 16

Completion Time

Process	Completion Time
P1	16
P2	7
P3	5
P4	11

Turnaround Time

$$TAT = CT - AT$$

Process	CT	AT	TAT
P1	16	0	16
P2	7	2	5
P3	5	4	1
P4	11	5	6

Waiting Time

$$WT = TAT - BT$$

Process	TAT	BT	WT
P1	16	7	9
P2	5	4	1
P3	1	1	0
P4	6	4	2

Average Waiting Time

$$\frac{9 + 1 + 0 + 2}{4} = \frac{12}{4} = \boxed{3}$$

---

7. Difference Between SJF and SRTF

SJF (Non-Preemptive)	SRTF (Preemptive)
Process cannot be interrupted.	Process can be interrupted.
Selects smallest burst time.	Selects smallest remaining time.
Once started, process runs until completion.	A newly arrived shorter process can preempt the current process.
Simpler to implement.	More complex due to context switching.
Optimal when all processes are ready simultaneously.	Optimal when processes arrive at different times.

---

8. Main Problem with SJF

The major problem with SJF is:

It is difficult to know the exact length of the next CPU burst in advance.

The operating system usually cannot know exactly how much CPU time a process will require.

Therefore, in practical systems, the CPU burst time is often **estimated using previous CPU bursts**.

---

9. Advantages of SJF

1. Gives minimum average waiting time when burst times are known.
2. Short processes are completed quickly.
3. Improves throughput.
4. SRTF can provide better response for short processes.

---

10. Disadvantages of SJF

1. Difficult to know the next CPU burst exactly.
2. Long processes may suffer from **starvation** if short processes continuously arrive.
3. Preemptive SJF causes more context switching.
4. More complex to implement than FCFS.

Important Formulas

$$\boxed{TAT = CT - AT}$$
$$\boxed{WT = TAT - BT}$$

For a non-preemptive process:

$$\boxed{WT = Start\,Time - Arrival\,Time}$$

Quick Exam Summary

Shortest-Job-First (SJF) scheduling selects the process with the smallest CPU burst time. It can be **non-preemptive**, where the running process cannot be interrupted, or **preemptive**, called **Shortest-Remaining-Time-First (SRTF)**, where a process with a shorter remaining time can preempt the currently running process. SJF gives the minimum average waiting time when all processes are ready simultaneously, but its main disadvantage is that the exact CPU burst time is difficult to predict.

Priority-Based Scheduling – Detailed Notes

1. What is Priority-Based Scheduling?

In **Priority Scheduling**, each process is assigned a **priority number**.

The CPU is allocated to the process with the **highest priority**.

In your notes:

Smaller priority number = Higher priority

For example:

Process	Priority
P1	3
P2	1
P3	4

Here, **P2 has the highest priority** because priority number **1 is the smallest**.

So the execution order will be:

P2 → P1 → P3

---

2. Important Point About Priority Numbers

There are two possible conventions:

Convention 1

Smaller number → Higher priority

Example:

- Priority 1 → Highest
- Priority 5 → Lowest

Convention 2

Larger number → Higher priority

Example:

- Priority 5 → Highest
- Priority 1 → Lowest

⚠️ **Always check the question carefully**.

Your first example uses:

Smallest integer = highest priority

But your Problem 1 specifically says:

Higher number represents higher priority

So for Problem 1, we must choose the **largest priority number first**.

---

3. Types of Priority Scheduling

Priority Scheduling can be:

A. Non-Preemptive Priority Scheduling

Once a process gets the CPU, it continues until completion.

Even if a higher-priority process arrives, the currently running process is **not interrupted**.

Example

Suppose:

Process	Arrival Time	Burst Time	Priority
P1	0	8	2
P2	2	2	5

Assume **higher number = higher priority**.

At time 0, only P1 is available, so P1 starts.

At time 2, P2 arrives with a higher priority.

However, because scheduling is **non-preemptive**, P1 continues.

| P1 | P2 |

0 8 10

P2 gets the CPU only after P1 finishes.

B. Preemptive Priority Scheduling

In preemptive priority scheduling, a currently running process can be interrupted if a new process with a higher priority arrives.

Using the same example:

Process	AT	BT	Priority
P1	0	8	2
P2	2	2	5

At time 0, P1 starts.

At time 2, P2 arrives with higher priority.

Therefore, P1 is interrupted.

| P1 | P2 | P1 |

0 2 4 10

---

4. Example from Your Notes

Given:

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

According to your notes:

Smaller priority number = Higher priority

Therefore, arrange the processes from smallest priority number to largest:

Execution Order	Process	Priority
1	P2	1
2	P5	2
3	P1	3
4	P3	4
5	P4	5

Therefore:

P2 → P5 → P1 → P3 → P4

Gantt Chart

| P2 | P5 | P1 | P3 | P4 |

0 1 6 16 18 19

The process with priority **1** executes first because it has the highest priority.

---

5. Problem 1 – Non-Preemptive Priority Scheduling

The table in the question appears to be:

Process	Arrival Time	Burst Time	Priority
P1	0	4	2

Process	Arrival Time	Burst Time	Priority
P2	1	3	1
P3	2	4	3
P4	3	5	4
P5	4	2	5

The question states:

Higher number represents higher priority.

Therefore:

Priority 5 > Priority 4 > Priority 3 > Priority 2 > Priority 1

We use **Non-Preemptive Priority Scheduling**.

Step 1: Time = 0

Only P1 has arrived.

Therefore, P1 gets the CPU.

Since this is non-preemptive, it executes until completion.

P1: 0 → 4

Step 2: Time = 4

By time 4, the following processes are available:

Process	Priority
P2	1
P3	3
P4	4
P5	5

Since a higher number means higher priority:

P5 has the highest priority.

Therefore:

P5: 4 → 6

Step 3: Time = 6

Remaining processes:

Process	Priority
P2	1
P3	3
P4	4

P4 has the highest priority.

P4: 6 → 11

Step 4: Time = 11

Remaining:

Process	Priority
P2	1
P3	3

P3 has the higher priority.

P3: 11 → 15

Step 5: Time = 15

Only P2 remains.

P2: 15 → 18

Final Gantt Chart

| P1 | P5 | P4 | P3 | P2 |

0 4 6 11 15 18

Completion Time

Process	AT	BT	Priority	CT
P1	0	4	2	4
P2	1	3	1	18
P3	2	4	3	15
P4	3	5	4	11
P5	4	2	5	6

Turnaround Time

$$TAT = CT - AT$$

Process	CT	AT	TAT
P1	4	0	4
P2	18	1	17
P3	15	2	13
P4	11	3	8
P5	6	4	2

Average Turnaround Time

$$\frac{4 + 17 + 13 + 8 + 2}{5} = \frac{44}{5} = \boxed{8.8}$$

Waiting Time

$$WT = TAT - BT$$

Process	TAT	BT	WT
P1	4	4	0
P2	17	3	14
P3	13	4	9

Process	TAT	BT	WT
P4	8	5	3
P5	2	2	0

Average Waiting Time

$$\frac{0 + 14 + 9 + 3 + 0}{5} = \frac{26}{5} = \boxed{5.2}$$

---

6. Problem: Starvation (Indefinite Blocking)

The major problem with Priority Scheduling is **Starvation**, also called **Indefinite Blocking**.

What is Starvation?

A low-priority process may wait for a very long time or may even **never get the CPU**.

Example

Suppose P1 has low priority.

P1 is waiting...

Before P1 gets the CPU, another high-priority process arrives.

The high-priority process is executed first.

Then another high-priority process arrives.

Again, P1 must wait.

High Priority → High Priority → High Priority → ...

↓

P1 keeps waiting

If high-priority processes keep arriving continuously, P1 may never execute.

This is called:

Starvation or Indefinite Blocking

---

7. Solution: Aging

The solution to starvation is called **Aging**.

What is Aging?

As a process waits in the ready queue for a long time, its priority is gradually increased.

Eventually, the process becomes high enough in priority to receive the CPU.

Example

Suppose:

P1 Priority = 1

After waiting for some time:

Priority = 2

After more waiting:

Priority = 3

Eventually, its priority becomes sufficiently high and it gets CPU access.

Aging prevents starvation by increasing the priority of processes that have waited for a long time.

---

8. Priority Assignment

Priorities can be assigned based on different factors.

A. Internal Factors

These are factors related to the characteristics of the process.

Examples include:

1. Timing Constraints

Processes that must complete quickly may receive higher priority.

2. Memory Requirements

The amount of memory required by a process may affect its priority.

3. I/O Burst to CPU Burst Ratio

Processes that perform frequent I/O operations may be assigned different priorities.

For example, an interactive I/O-bound process may receive higher priority to improve responsiveness.

B. External Factors

These are factors outside the technical characteristics of the process.

Examples include:

1. Importance of the Process

A critical system process may receive higher priority.

2. Financial Considerations

In some systems, processes from users or services with higher importance may receive higher priority.

3. Hierarchy Among Users

Different users or groups may be assigned different priority levels.

Quick Comparison

Feature	Priority Scheduling
Selection Basis	Priority number
Highest Priority	Depends on system convention
Types	Preemptive and Non-preemptive
Main Problem	Starvation
Solution	Aging

Exam Summary

Priority-Based Scheduling assigns a priority number to each process, and the CPU is allocated to the process with the highest priority. Priority scheduling can be **preemptive**, where a running process can be interrupted by a higher-priority process, or **non-preemptive**, where a process continues until completion. Its main disadvantage is **starvation**, where low-priority processes may wait indefinitely. **Aging** is used to prevent starvation by gradually increasing the priority of processes that wait for a long time.

Yes. Using the same question, but with **Preemptive Priority Scheduling**.

Given

Higher number = higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	3	1
P3	2	4	3
P4	3	5	4
P5	8	2	5

Step-by-step scheduling

Time 0 to 1

Only P1 has arrived.

P1 runs from 0 to 1

Remaining burst time of P1 = **3**.

Time 1: P2 arrives

- P1 priority = 2
- P2 priority = 1

P1 has higher priority, so it continues.

P1 continues from 1 to 2

Remaining burst time of P1 = **2**.

Time 2: P3 arrives

- P1 priority = 2
- P3 priority = 3

P3 has higher priority, so **P1 is interrupted**.

P1: 0 → 2

P3 starts at 2

Time 3: P4 arrives

- P3 priority = 3
- P4 priority = 4

P4 has higher priority, so **P3 is interrupted**.

P3: 2 → 3

P4 starts at 3

P3 has executed for 1 unit, so its remaining burst time is:

$$4 - 1 = 3$$

Time 3 to 8

P4 continues executing.

At time 8, P5 arrives.

P4 has executed from 3 to 8 = **5 units**.

Since P4's burst time is 5, it **completes exactly at time 8**.

So there is no interruption here.

Time 8: P5 arrives

Ready processes are:

Process	Remaining Burst	Priority
P1	2	2
P2	3	1
P3	3	3
P5	2	5

P5 has the highest priority.

P5: 8 → 10

P5 completes.

Time 10

Remaining processes:

Process	Remaining Burst	Priority
P1	2	2
P2	3	1
P3	3	3

P3 has the highest priority.

P3: 10 → 13

P3 completes.

Time 13

Remaining:

Process	Remaining Burst	Priority
P1	2	2
P2	3	1

P1 has higher priority.

P1: 13 → 15

Time 15

Only P2 remains.

P2: 15 → 18

Final Gantt Chart

| P1 | P3 | P4 | P5 | P3 | P1 | P2 |

0 2 3 8 10 13 15 18

Completion Time (CT)

Process	AT	BT	Priority	CT
P1	0	4	2	15
P2	1	3	1	18
P3	2	4	3	13
P4	3	5	4	8
P5	8	2	5	10

Turnaround Time

$$TAT = CT - AT$$

Process	TAT
P1	15 − 0 = 15
P2	18 − 1 = 17
P3	13 − 2 = 11
P4	8 − 3 = 5
P5	10 − 8 = 2

Average Turnaround Time

$$\frac{15 + 17 + 11 + 5 + 2}{5} = \boxed{10}$$

Waiting Time

$$WT = TAT - BT$$

Process	TAT	BT	WT
P1	15	4	11
P2	17	3	14
P3	11	4	7
P4	5	5	0
P5	2	2	0

Average Waiting Time

$$\frac{11 + 14 + 7 + 0 + 0}{5} = \frac{32}{5} = \boxed{6.4}$$

Final Answers

$$\boxed{\text{Average Waiting Time} = 6.4}$$

$$\boxed{\text{Average Turnaround Time} = 10}$$

The main difference from **non-preemptive priority scheduling** is that **P1 is interrupted by P3 at time 2, and P3 is interrupted by P4 at time 3**, because higher-priority processes arrive. **Round Robin (RR) Scheduling – Detailed Notes**

1. What is Round Robin Scheduling?

Round Robin (RR) is a **preemptive CPU scheduling algorithm** mainly used in **time-sharing systems**.

Each process is given a fixed small amount of CPU time called the **Time Quantum (q)** or **Time Slice**.

After using the CPU for one time quantum:

- If the process is completed, it leaves the system.
- If it is not completed, it is **preempted**.
- It is placed at the **end of the ready queue**.
- The next process gets the CPU.

Simple idea

P1 → P2 → P3 → P4 → P1 → P2 → ...

Each process gets a turn.

---

2. Time Quantum

The **Time Quantum (q)** is the maximum amount of CPU time given to a process in one turn.

For example:

Time Quantum = 2 units

A process can execute for a maximum of 2 units before the CPU is given to the next process.

Example

Suppose:

P1 Burst Time = 5

Time Quantum = 2

P1 cannot execute for all 5 units at once.

It executes as:

First turn: 2 units → Remaining = 3

Second turn: 2 units → Remaining = 1

Third turn: 1 unit → Completed

---

3. Working of Round Robin

The ready queue works like a **circular FIFO queue**.

Steps

1. Take the first process from the ready queue.
2. Give it the CPU for one time quantum.
3. If the process completes, remove it.
4. If it does not complete, put it at the end of the ready queue.
5. Select the next process.
6. Repeat until all processes complete.

---

4. Important Rule About New Processes

Newly arriving processes are added to the **end of the ready queue**.

Similarly, processes that finish their I/O operation and become ready are also added to the **end of the ready queue**.

Example:

Initially:

Ready Queue: P1 → P2 → P3

P1 gets the CPU.

While P1 is running, P4 arrives:

Ready Queue: P2 → P3 → P4

When P1's quantum expires and it is not complete, it goes to the end:

Ready Queue: P2 → P3 → P4 → P1

---

5. Maximum Waiting Time

If there are **n processes** in the ready queue and the time quantum is **q**, a process will wait at most approximately:

$$\boxed{(n-1)q}$$

time units before getting another turn.

This is because every other process gets one quantum before it gets the CPU again.

---

6. Effect of Time Quantum

The value of the time quantum greatly affects Round Robin performance.

When Time Quantum is Large

If:

q is very large

then processes may finish before their quantum expires.

Therefore, Round Robin behaves similar to:

$$\boxed{FCFS}$$

Example

Burst Times: P1 = 5, P2 = 3

Time Quantum = 100

Both processes complete in their first turn.

Execution:

P1 → P2

This is essentially FCFS.

When Time Quantum is Very Small

If the quantum is very small, processes are switched very frequently.

Example:

P1 → P2 → P3 → P1 → P2 → P3 → ...

This makes the system appear as though each process has its own CPU.

This is useful in **time-sharing and interactive systems** because every process gets a quick response.

However, extremely small quantum values cause many **context switches**, creating overhead.

---

7. Example: Round Robin

Given:

Process	Burst Time
P1	53
P2	17
P3	68
P4	24

Time Quantum:

$$q = 20$$

Assume all processes arrive at time 0.

Initial ready queue:

P1 → P2 → P3 → P4

First Round

P1

Burst = 53

Runs for 20.

0 → 20

Remaining:

$$53 - 20 = 33$$

P1 goes to the end of the queue.

P2

Burst = 17, which is less than 20.

Therefore, P2 completes.

20 → 37

P3

Burst = 68.

Runs for 20.

37 → 57

Remaining:

$$68 - 20 = 48$$

P4

Burst = 24.

Runs for 20.

57 → 77

Remaining:

$$24 - 20 = 4$$

Second Round

P1

Remaining = 33.

Runs for 20.

77 → 97

Remaining:

$$33 - 20 = 13$$

P3

Remaining = 48.

Runs for 20.

97 → 117

Remaining:

$$48 - 20 = 28$$

P4

Remaining = 4.

P4 completes.

117 → 121

Third Round

P1

Remaining = 13.

P1 completes.

121 → 134

P3

Remaining = 28.

Runs for 20.

134 → 154

Remaining:

$$28 - 20 = 8$$

Final Round

P3

Remaining = 8.

P3 completes.

154 → 162

Final Gantt Chart

| P1 | P2 | P3 | P4 | P1 | P3 | P4 | P1 | P3 | P3 |

0 20 37 57 77 97 117 121 134 154 162

Waiting Time Formula for Round Robin

Instead of adding all the separate waiting periods, the easiest method is:

$$\boxed{WT = TAT - BT}$$

where:

$$\boxed{TAT = CT - AT}$$

Since all processes arrive at time 0:

$$TAT = CT$$

Calculate Waiting Time

Process	AT	BT	CT	TAT	WT
P1	0	53	134	134	81
P2	0	17	37	37	20
P3	0	68	162	162	94
P4	0	24	121	121	97

Average Waiting Time

$$\frac{81 + 20 + 94 + 97}{4}$$

$$= \frac{292}{4}$$

$$\boxed{73}$$

---

8. Time Quantum = 30

For the same processes:

Process	Burst Time
P1	53
P2	17
P3	68
P4	24

$$q = 30$$

The Gantt chart is:

| P1 | P2 | P3 | P4 | P1 | P3 | P3 |

0 30 47 77 101 124 154 162

The average waiting time is:

$$\boxed{68}$$

This shows that changing the time quantum can change the average waiting time.

---

9. Advantages of Round Robin

1. Fair scheduling — every process gets a CPU turn.
2. No starvation — every ready process eventually gets the CPU.
3. Good response time.
4. Suitable for time-sharing systems.
5. No process can monopolize the CPU for a long time.

---

10. Disadvantages of Round Robin

1. Performance depends heavily on the time quantum.
2. Very small quantum causes many context switches.
3. Very large quantum makes RR behave like FCFS.

4. Average turnaround time may be higher than SJF.
5. Context switching creates CPU overhead.

---

11. FCFS vs RR

FCFS	**Round Robin**
Non-preemptive	Preemptive
Process runs until completion	Process runs for one time quantum
Simple FIFO execution	Circular FIFO execution
Poor response time for long processes	Better response time
Suitable for batch systems	Suitable for time-sharing systems

Most Important Exam Point

Round Robin Rule

A process executes for a maximum of one time quantum. If it is not completed, it is preempted and placed at the end of the ready queue.

Formula

$$\boxed{TAT = CT - AT}$$
$$\boxed{WT = TAT - BT}$$

Time Quantum Effect

$$\boxed{q \text{ large} \Rightarrow RR \approx FCFS}$$
$$\boxed{q \text{ small} \Rightarrow Betterresponsetimebutmorecontextswitching}$$

For your **Problem 1 and Problem 2**, the table formatting is slightly unclear. If you send the arrival time and burst time tables clearly (or a screenshot), I can solve both step-by-step with the exact **ready queue changes, Gantt chart, waiting time, turnaround time, and averages**.